Week 1 - Domain 1: SDLC Automation - Amazon CodeGuru Reviewer and Profiler

Study week: Oct 19 - Oct 25, 2026 **Exam:** AWS DOP-C02 (Domain 1 = 22% of the exam) **Depth:** light review

Section	What it covers
7.1	Reviewer vs Profiler side by side
7.2	CodeGuru Reviewer: findings, review types, repositories, setup
7.3	CodeGuru Profiler: agent, views, recommendations, anomalies
7.4	CodeGuru vs X-Ray, Inspector, DevOps Guru, CloudWatch
7.5 - 7.7	Exam traps, hands-on lab, check yourself

Part 7: Amazon CodeGuru

Availability note: AWS has been moving many CodeGuru features into **Amazon Q Developer** (code reviews, security scans) and **Amazon Inspector** (code vulnerability scanning). **CodeGuru Security** (a separate product) has been retired, and **CodeGuru Reviewer** may no longer accept new customers. Check the current AWS docs. For the exam, know **what each tool does and when to choose it** over similar services.

7.1 The two services side by side

	CodeGuru Reviewer	CodeGuru Profiler
What it does	Reviews code automatically without running it (machine learning + automated reasoning)	Measures a running application - which code uses the most CPU, time and memory
When it works	Before deployment: pull requests and full repository scans	After deployment: test or production
Languages	Java and Python	JVM languages (Java, Kotlin, Scala) and Python
Output	Recommendations posted as comments on the pull request	Flame graphs, recommendation reports, anomaly alerts
Main benefit	Code quality and security; AWS API misuse; hard-coded secrets	Faster code and lower compute cost - the most expensive lines of code
Exam keywords	automated code review, pull request, before merging	production performance, CPU / latency hot spots, reduce compute cost

7.2 CodeGuru Reviewer

What it finds:

- **AWS API misuse:** not handling **pagination**, missing **retries / backoff**, re-creating SDK clients, not checking whether a call failed.
- **Concurrency problems:** race conditions, unsafe sharing of data between threads.
- **Resource leaks:** streams, connections or files never closed.
- **Security issues:** OWASP Top 10 / CWE types - injection, weak encryption, missing input validation, logging sensitive data.
- **Hard-coded secrets:** passwords, API keys, AWS keys, database credentials - suggests moving them to **AWS Secrets Manager**.
- **Code quality:** duplicated code, hard-to-maintain code, Python best practices.

Two kinds of review:

Type	When it runs	What it scans
Incremental (pull request) review	Automatically on every pull request in an associated repository	Only the changed code; comments posted on the pull request
Full repository analysis	When you start it (console, CLI, API) for a branch	The whole branch - good for a first baseline

Supported repositories:

- **CodeCommit**, **GitHub** (incl. GitHub Enterprise Cloud and Server), **Bitbucket** - GitHub and Bitbucket through **CodeConnections**.
- **S3, for CI/CD:** upload the source (and for Java the **build artifacts** - `.jar` / `.class`) to a bucket whose name **must start with** `codeguru-reviewer-`, then start the review with the CLI / API. This is how CodeBuild, Jenkins and other CI tools use it.
- **GitHub Actions:** an official CodeGuru Reviewer action runs it inside a workflow.

Setup and other points:

- **Associate the repository** once; pull request reviews then happen automatically.
- **Java security scans need the build artifacts**, not only the source.
- Encryption: AWS-owned key by default, or a **customer-managed KMS key** for the repository association.
- Service-linked role: `AWSServiceRoleForAmazonCodeGuruReviewer`.
- Developers can rate recommendations (thumbs up / down).
- **Pricing:** by **lines of code** analysed per month; free trial for new users.

7.3 CodeGuru Profiler

How it works:

- A lightweight **agent** inside the application **samples** what the code is doing (about **1% overhead**) and sends data to a **profiling group** (one application or service).
- **Runs on:** EC2, ECS, EKS, Fargate, **Lambda**, and **on-premises** (if the agent can reach the Profiler API).
- **Adding the agent:**
  - **Java:** `-javaagent:codeguru-profiler-java-agent.jar`, or start it from code.
  - **Python:** the `codeguru_profiler_agent` package, started in code or on the command line.
  - **Lambda (Java / Python):** turn on **"Code profiling"** in the function settings - adds the agent layer and environment variables.
- **IAM:** the application role needs `AmazonCodeGuruProfilerAgentAccess` (`codeguru-profiler:ConfigureAgent`, `PostAgentProfile`) for its profiling group.

What you see:

View	Shows
CPU usage flame graph	Which methods use the most CPU (wider bars use more)
Latency (wall-clock) view	Where time is spent including waiting (I/O, network, locks)
Heap summary (Java)	Memory use by object type - memory growth or leaks
Hotspot and inspect views	Zoom into the most expensive call paths

Recommendations and anomalies:

- **Recommendation reports** for known costly patterns: re-creating AWS SDK clients per call, re-compiling the same regex, heavy logging / string formatting, slow serialization - often with an **estimated compute cost saving**.
- **Anomaly detection:** learns normal behaviour and **alerts (e.g. SNS)** when a method suddenly uses much more CPU or time.
- **Pricing:** by **profiling (sampling) hours**, with a free tier.

7.4 CodeGuru compared with similar services

Need in the question	Service
Review code in pull requests for bugs, AWS best practices, hard-coded secrets (Java / Python)	CodeGuru Reviewer (or Amazon Q Developer code review)
Most expensive methods / lines in a running app, reduce CPU cost	CodeGuru Profiler
Follow one request across several services, find the slow service	AWS X-Ray (or CloudWatch Application Signals)
Known vulnerabilities in packages, images or Lambda code	Amazon Inspector (and ECR enhanced scanning)
Operational anomalies (unusual errors / latency) across resources, ML insights	Amazon DevOps Guru
Server / container metrics (CPU %, memory %)	CloudWatch (metrics, Container Insights, Lambda Insights)
Secrets found in code must be stored and rotated	AWS Secrets Manager

Typical exam scenarios:

- Automated review comments on every PR in a CodeCommit / GitHub Java repo, including SDK misuse and hard-coded credentials = **CodeGuru Reviewer** (associate the repository).
- Code review as part of a CodeBuild pipeline = upload code + build artifacts to `codeguru-reviewer-*` bucket and start the review through the API (or the GitHub Action).
- Production Java service uses too much CPU - find what to optimise = **CodeGuru Profiler**.
- Slow Python Lambda - find the slow code = turn on **code profiling** (Profiler).
- Slow API calling 5 microservices - which one is the bottleneck = **X-Ray**, not Profiler.
- Dependencies with known CVEs = **Inspector**, not CodeGuru.

7.5 Exam traps: quick table

Scenario	Answer
Automated pull request code review (Java / Python)	CodeGuru Reviewer - associate the repository
Scan the whole repository once for a baseline	Reviewer full repository analysis
Code review from CodeBuild or another CI tool	S3 bucket named codeguru-reviewer-* + API (or GitHub Action)
Java security findings missing	Provide the build artifacts (.jar / .class) with the source
Hard-coded passwords found in code	Reviewer detects them - move to Secrets Manager
CPU hot spots in running code	CodeGuru Profiler (flame graphs)
Time spent waiting on I/O or locks	Profiler latency (wall-clock) view
Java memory keeps growing	Profiler heap summary
Profile a Lambda function	Turn on Code profiling in the Lambda settings
Profiler agent can't send data	Role needs AmazonCodeGuruProfilerAgentAccess for the profiling group
Alert when a method suddenly gets slower	Profiler anomaly detection (SNS)
Bottleneck between microservices	X-Ray, not Profiler
Package or image CVEs	Inspector, not CodeGuru
Operational anomalies across resources	DevOps Guru
Go, Node.js or .NET code	Not supported - CodeGuru is Java and Python only

7.6 Hands-on lab (optional, about 45 minutes)

Check first that the services are still available in your account and Region.

1. **Reviewer:** associate your CodeCommit or GitHub repository; open a pull request that adds a hard-coded password and an S3 `list_objects` call that ignores pagination; read the comments on the pull request.

2. **Profiler:** create a Java or Python Lambda function with a CPU-heavy loop; turn on **Code profiling**; invoke it several times; open the profiling group and read the **flame graph**.
3. Clean up: disassociate the repository and delete the profiling group.

7.7 Check yourself

1. Which CodeGuru service comments on pull requests, and which analyses a running application?
2. Which languages does CodeGuru support?
3. How do you run CodeGuru Reviewer from a CodeBuild pipeline?
4. A Python Lambda function is slow. How do you find the slow lines of code?
5. A request through 6 microservices is slow. Is CodeGuru Profiler the right tool?

Answers:

1. **Reviewer** comments on pull requests (static analysis). **Profiler** analyses the running application.
2. **Java and Python** (Profiler: all JVM languages - Java, Kotlin, Scala - and Python).
3. Upload source (and Java build artifacts) to an S3 bucket named `codeguru-reviewer-*` and start the review with the CLI / API, or use the GitHub Action.
4. Turn on **Code profiling** for the function (CodeGuru Profiler) and read the flame graph.
5. **No** - use **X-Ray** to find the slow service, then Profiler inside that service if needed.